

Can Agents Design Better Chips with a Higher Level Abstraction?

(Invited Paper)

Zijian Ding
UCLA
Los Angeles, CA, USA
bradyd@cs.ucla.edu

Yizhou Sun
UCLA
Los Angeles, CA, USA
yzsun@cs.ucla.edu

Yang Zou
UCLA
Los Angeles, CA, USA
willz0123@ucla.edu

Jason Cong
UCLA
Los Angeles, CA, USA
cong@cs.ucla.edu

Abstract

Large Language Model (LLM) agents are increasingly being explored for chip design, but most existing approaches operate directly at RTL. We ask whether agents can design better chips by leveraging higher-level abstractions. We compare Direct RTL Design, Agent-based HLS Design, Post-Compiler HLS Refinement, and Post-HLS RTL Refinement, and combine Agent-based HLS Design with Post-HLS RTL Refinement as Agent-based HLS with RTL Refinement (AHRR). We use FPGAs as a practical, easy-to-deploy platform for end-to-end evaluation, but note that the design-flow tradeoffs we study are largely independent of the target technology. Across a diverse 11-tasks benchmark suite, AHRR achieves a $2.6\times$ geometric-mean speedup over Direct RTL Design across our benchmark suite. Case studies show that HLS distills design knowledge into abstractions that agents can leverage, while RTL refinement recovers lower-level optimization opportunities. Together, these results make AHRR a promising workflow for agent chip design. The code and evaluation artifacts are available at <https://github.com/ZijD/AHRR>.

CCS Concepts

• **Hardware** → **Electronic design automation**.

Keywords

High-level synthesis, LLM agents, FPGA

ACM Reference Format:

Zijian Ding, Yang Zou, Yizhou Sun, and Jason Cong. 2026. Can Agents Design Better Chips with a Higher Level Abstraction?: (Invited Paper). In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD '26)*, November 08–12, 2026, San Jose, CA, USA. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3831252.3847810>

1 Introduction

Recent advances in Large Language Models (LLMs) and LLM-based coding agents have created growing interest in applying them to

chip design. LLM agents have been explored for hardware generation, debugging, verification and optimization [1, 10, 13, 17, 34], and it is already applied in commercial design [16].

However, RTL is not the only interface between an agent and the hardware design process. It has been widely used in manual designs as it provides cycle-accurate specification. In contrast, High-Level Synthesis (HLS) [7, 8] lets designers express computation in C/C++ while delegating scheduling, pipelining, and RTL generation to a compiler, and domain-specific HLS frameworks [2, 5, 24] raise the abstraction further by providing and optimizing parameterized architecture templates. Agents have begun to work at the HLS level as well [19, 23, 35], but whether the abstraction itself helps an agent to produce a higher quality design is less clear. Direct RTL generation gives the agent full control, whereas HLS and domain-specific compilers restrict the design space to a usually high-quality subset. The two approaches can also be combined in multiple ways: (i) Direct RTL Design, where an agent creates RTL directly, (ii) Agent-based HLS Design, where an agent optimized HLS C/C++ code and then relies on HLS tools to generate RTL, (iii) Post-HLS RTL Refinement, where an agent further optimizes the HLS-generated RTL, and (iv) Post-Compiler HLS Refinement, where a domain-specific compiler first generates an HLS design that is further refined by an agent, with the resulting RTL serving as the input to flow (iii). We refer to flows (ii)+(iii) as AHRR: Agent-based HLS with RTL Refinement (Figure 7), where the agent first designs at the HLS level and then refines the generated RTL.

These alternatives points to a fundamental question: *can agents design better chips when operating at a higher level of abstraction using HLS, and how?*

Our contributions are:

- We systematically study LLM-based chip-design agents across RTL, HLS, and compiler-assisted design flows.
- We show our proposed AHRR outperform Direct RTL by $2.6\times$ geometric-mean.
- We further study Post-Compiler HLS Refinement, showing that agents can improve both HLS-generated RTL and compiler-generated HLS.

Evaluation across these design flows (Figure 1) show that agents can design better chips by leveraging HLS and domain-specific HLS compilers that encode human design knowledge, combining machine and human intelligence.



This work is licensed under a Creative Commons Attribution 4.0 International License. ICCAD '26, San Jose, CA, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2873-0/2026/11

<https://doi.org/10.1145/3831252.3847810>

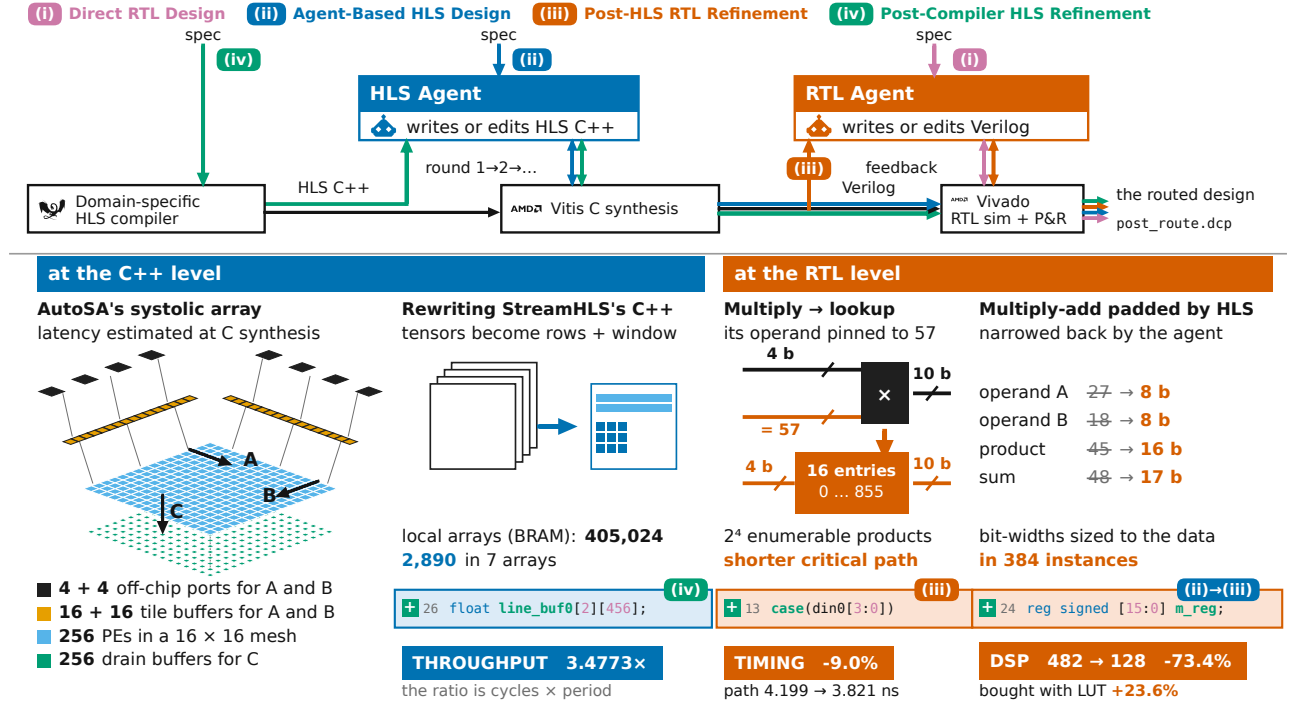


Figure 1: The four design flows studied in this paper. In (i) Direct RTL Design, the agent writes RTL directly from the specification. In (ii) Agent-based HLS Design, the agent writes HLS C++ and Vitis HLS generates RTL. In (iii) Post-HLS RTL Refinement, the agent further refines HLS-generated RTL. In (iv) Post-Compiler HLS Refinement, a domain-specific compiler generates the initial HLS C++, which the agent further refines.

2 Background

HLS raises hardware design from RTL to C/C++ and reduces the verification burden [7, 8]. Most agentic chip-design work targets RTL [10, 13, 17, 22, 34], while recent work explores HLS for pragma insertion, refactoring, and design-space exploration [6, 19, 23, 25, 26, 35]. Domain-specific HLS compilers generate optimized architectures for particular computation patterns [2, 4, 5, 18, 24, 29–31]. We use AutoSA and StreamHLS as starting points. Our agents follow the standard iterative coding-agent loop [15, 21, 28], under a fixed round budget and a hidden testbench.

3 Methodology

In this section, we describe the common agent loop, the four design flows in Figure 1, and the hidden testbench.

Specification-driven design. In Direct RTL Design (i) and Agent-based HLS Design (ii), the agent starts from the same task specification, task package, and evaluation harness. The task package includes a correct, unoptimized C++ implementation, one test stimulus, and a fixed interface contract. The agent then designs the accelerator either directly in RTL or in HLS C++. Within each round, the agent has up to one hour to develop and submit a candidate design. The evaluation harness records correctness on the hidden testbench, simulated cycle count, post-route clock period, and resource utilization. We use the open-source coding agent pi [14] without modification. The agent’s session is carried across rounds, and each round’s prompt reports the previous three rounds of evaluation results. We keep the prompt minimal, specifying only the

optimization objective, resource budget, interface requirements, and submission protocol. In the RTL flow, the agent additionally receives the required port specification and, for AXI-based tasks, a provided AXI wrapper around the kernel.

Refinement from tool-generated designs. Flows (iii) and (iv) start from an existing implementation rather than only the task specification. In Post-Compiler HLS Refinement (iv), the agent starts from HLS C++ generated by a domain-specific compiler, such as AutoSA [24] or StreamHLS [2]. In Post-HLS RTL Refinement (iii), it starts from Verilog produced by Vitis HLS, either from an Agent-based HLS design or a refined compiler-generated HLS design. Apart from the starting artifact and abstraction level, the optimization loop and evaluation procedure remain unchanged.

Hidden testbench. The public testbench provides a single stimulus for local development, while correctness is evaluated with a hidden testbench containing broader inputs and specification-derived corner cases. We strengthen each testbench until the C++ reference reaches 100% branch coverage, and additionally measure statement and branch coverage on the reference RTL implementation.

4 Evaluation

In this section, we evaluate how the abstraction level affects agentic chip design. We first compare Direct RTL Design and Agent-based HLS Design to isolate the benefit of HLS. We then evaluate Post-HLS RTL Refinement and Post-Compiler HLS Refinement.

Table 1: The 11-task benchmark suite spans LLM inference, agentic memory, robotics, and GEMM. Selected tasks use fixed-point arithmetic to avoid floating-point IP. Dynamic dimensions are preserved when possible, and some workloads are scaled down to reduce RTL simulation time.

Task	Category	Interface	Arithmetic	Dynamic axis	Scaled down
LLM1 llm_decode_attention_kv	LLM decode attention over a KV cache [9]	m_axi	fixed	seq_len	no
LLM2 llm_norm_quant_linear	fused RMSNorm + quantize + INT8 linear [32]	m_axi	mixed	seq_len	yes - 12 tokens
GEM1 gemm_i8_512	dense INT8 GEMM, two variants: 3/12 AXI	m_axi	fixed	none	no
MEM1 mem_bm25_gather_score	BM25 retrieval ranking over postings [20]	ap_memory	fixed	none	no
MEM2 mem_kv_page_compact	paged-KV [11] Prepare Memory stage	ap_memory	fixed	seq_len	no
MEM3 mem_lserve_page_select	LServe-style page selection [27]	ap_memory	fixed	none	no
ROB1 robo_nms_boxes	greedy non-maximum suppression	ap_memory	fixed	num_boxes	no
ROB2 robo_pillar_encoder	PointPillars pillar feature encoder [12]	ap_memory	fixed	n_points	yes - coarser 0.25 m pillar
ROB3 robo_stereo_sad	stereo block matching, SAD	ap_memory	fixed	none	yes - IMG_W/IMG_H scaled down
SHL1 streamhls_gemm	float32 GEMM, generated by StreamHLS [2]	ap_memory	float	none	no
SHL2 streamhls_dw_conv	dw-conv generated by StreamHLS	ap_memory	float	none	no

Table 2: Parameters fixed across experiments.

Parameter	Value
device	AMD Alveo U55C (xcu55c-fsvh2892-2L-e)
toolchain	Vitis 2025.2, out of context (no shell)
agents	gemini-3.1-pro (high), gpt-5.6-sol (xhigh)
maximum rounds	5
agent round timeout	60 min
evaluation timeout	RTL sim 75 min, syn + PnR 90 min
total timeout	10 h
correctness check	C sim + RTL sim
clock target	3.33 ns
achieved clock	max(target - WNS, target)
resource ceiling	60% of BRAM, DSP, FF, LUT, URAM

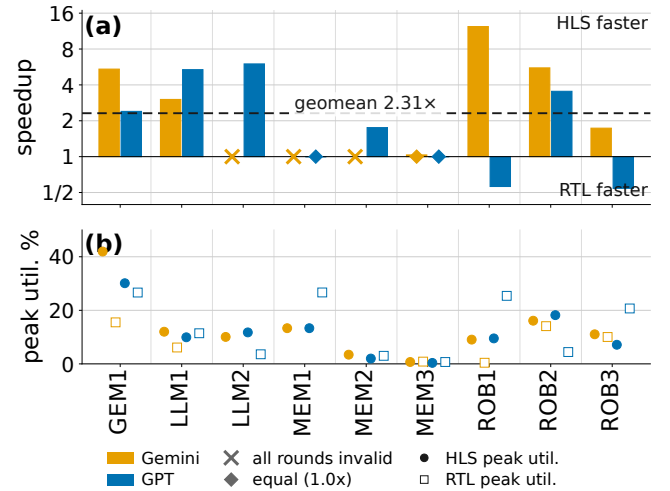
4.1 Experimental Setup

We run Gemini 3.1 Pro at high reasoning effort and GPT-5.6-sol at extra-high effort under the same pi harness with every parameter fixed (Table 2).

A design is valid if it passes the hidden testbench, routes successfully, and stays within the resource ceiling. We compute execution time as the simulated cycle count multiplied by the maximum of the post-route clock period and the target clock period. All reported speedups are measured relative to execution time on the same task. Each experiment runs once. Table 1 lists the eleven tasks. For GEM1, we have two variants, one with naive sequential design with 3 AXI masters (A,B,C) for RTL v.s. HLS comparison, and the other with generated by AutoSA with 12 AXI masters.

4.2 Overview: Design-Flow Comparison

Table 3 compares the specification-driven workflows. Agent-based HLS Design achieves a 2.31× geometric-mean speedup over Direct RTL Design. Extending it with Post-HLS RTL Refinement to form AHRR increases the speedup to 2.62×. We separately evaluate Post-Compiler HLS Refinement in Section 4.5.

**Figure 2: Performance and resource comparison of Agent-based HLS Design and Direct RTL Design. With HLS, state-of-the-art agents achieve a 2.31× geometric-mean speedup over direct RTL design.**

4.3 Agent-based HLS vs. Direct RTL

We first isolate the effect of abstraction by giving the same agent the same task specification and asking it to design in either RTL or HLS, corresponding to flows (i) and (ii).

With the HLS tool available, the agent can express parallelism through loop transformations and pragmas. In RTL, the same agent must realize these structures explicitly and often produces a narrower datapath. This demonstrates that HLS tools carry distilled design knowledge that agents can leverage. We provide a detailed case study in Figure 4.

Direct RTL Design may still win when the design is small. The two cases where RTL outperforms HLS are both on small kernels, ROB1 and ROB3, and by 1.78× and 1.85× respectively. When the

Table 3: Comparison of the specification-driven design workflows. Speedups are relative to Direct RTL Design on the same task and model. Geometric means include only pairs with a valid Direct RTL baseline; n denotes the number of included pairs. AHRR has $n = 13$ because Post-HLS RTL Refinement was not evaluated on the two GEM1 (3 AXI) cases.

Task	Model	(i) Direct RTL Design		(ii) Agent-based HLS Design		(ii)→(iii) AHRR	
		ms	speedup	ms	speedup	ms	speedup
GEM1 (3 AXI)	gemini	2.19	1.00×	0.40	5.43×	–	–
	gpt	1.31	1.00×	0.55	2.40×	–	–
LLM1	gemini	1.29	1.00×	0.43	3.02×	0.43	3.02×
	gpt	2.54	1.00×	0.47	5.37×	0.43	5.87×
LLM2	gemini	×	–	0.20	–	0.20	–
	gpt	1.23	1.00×	0.21	6.00×	0.21	6.00×
MEM1	gemini	×	–	7.44	–	7.44	–
	gpt	5.26	1.00×	5.27	1.00×	5.25	1.00×
MEM2	gemini	×	–	0.37	–	0.37	–
	gpt	0.65	1.00×	0.37	1.75×	0.32	2.02×
MEM3	gemini	3.62	1.00×	3.50	1.03×	3.50	1.03×
	gpt	3.50	1.00×	3.50	1.00×	3.50	1.00×
ROB1	gemini	6.16	1.00×	0.50	12.35×	0.45	13.82×
	gpt	0.30	1.00×	0.53	0.56×	0.33	0.90×
ROB2	gemini	35.38	1.00×	6.37	5.56×	6.22	5.69×
	gpt	9.22	1.00×	2.61	3.53×	2.20	4.19×
ROB3	gemini	0.02	1.00×	0.01	1.73×	0.01	1.95×
	gpt	0.01	1.00×	0.01	0.54×	0.00	2.09×
<i>geo. mean</i>		1.00× ($n=15$)		2.31× ($n=15$)		2.62× ($n=13$)	

kernel state fits in registers, the RTL flow can place it there directly, while HLS can only express the intent through binding pragmas. We also note that we use naive agent for both Agent-based HLS Design and Direct RTL Design, and a simple prompt update may significantly improve/degrade performance on both sides.

We next ask whether the agent can combine both advantages by refining the HLS-generated RTL.

4.4 Post-HLS RTL Refinement

We next apply Post-HLS RTL Refinement to the Verilog generated from the best Agent-based HLS design. The agent continues optimization directly at the RTL level.

Across the sixteen valid cases, Post-HLS RTL Refinement improves performance by 1.17× geometric mean over its HLS-generated RTL starting point.

Figure 3 isolates the effect of Post-HLS RTL Refinement, showing the per-round progress of each agent. We use throughput/latency as the primary objective and resource usage as the secondary objective. On LLM1, MEM1/2/3, and ROB2, agents make incremental improvements over the HLS-generated RTL. On ROB1 and ROB3, the agents achieve 2–4× speedups through complete rewrites, consistent with the results in Figure 2. On LLM2 and MEM3, GPT and Gemini also substantially reduce resource usage with only a few lines of RTL edits, which we examine in Section 5.3.

Most successful rounds make local edits rather than large rewrites, and these edits usually change the cycle count by less than 1.15×. Every larger latency improvement comes from a

Table 4: Post-Compiler HLS Refinement relative to the domain-specific compiler baseline, reported as Gemini / GPT. We use StreamHLS for SHL1/SHL2 and AutoSA for GEM1.

	SHL1	SHL2	GEM1
DS-compiler	StreamHLS	StreamHLS	AutoSA
Compiler baseline	1.0	1.0	1.0
Post-Compiler HLS Refinement	1.0 / 1.0	3.3 / 1.7	1.3 / 1.2

complete rewrite, where the agent discards the HLS-generated RTL and writes a new implementation.

4.5 Post-Compiler HLS Refinement

We finally study a stronger starting point, where a domain-specific compiler first generates the HLS design and the agent refines the generated C++. Table 4 reports the results. On SHL2, Gemini and GPT improve StreamHLS by 3.3× and 1.7×. On GEM1, both improve AutoSA by 1.3× and 1.2× while changing the cycle count by less than 2%, with the gains coming from improved timing. Gemini changes storage bindings (“bind_storage type=ram_2p impl=lutram”) and FIFO depths (2→4), while GPT restructures control and the PE datapath. In both cases, the agents incrementally improve an already optimized compiler-generated HLS design, showing that useful optimization opportunities remain after compiler optimization.

For SHL1, neither agent improves upon the compiler-generated design. This is not necessarily an intrinsic limitation of the agents. The initial compiler-generated design already requires simulation time close to our timeout limit, leaving little room for iterative

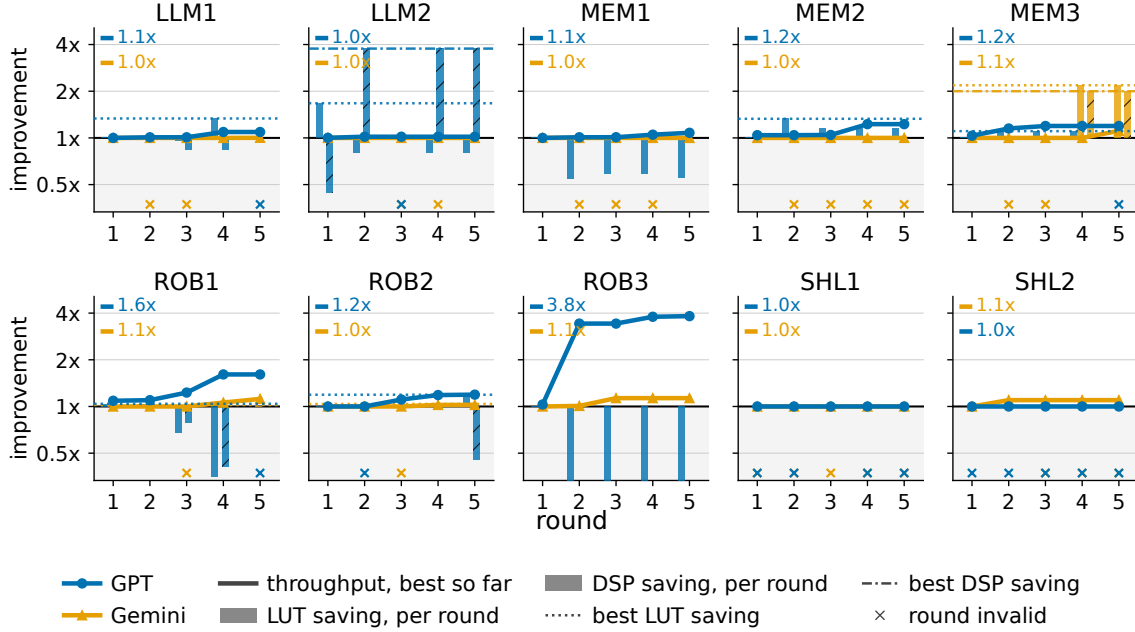


Figure 3: Per-round progress of Post-HLS RTL Refinement, normalized to the HLS-generated RTL that seeds each experiment. All axes are oriented so that higher is better. LUT and DSP savings are shown alongside speedup to capture the tradeoff between area and performance. We also include SHL1/SHL2 to illustrate that Post-HLS RTL Refinement becomes challenging when simulation time is long.

exploration. A better evaluation harness could use HLS latency estimates during optimization and reserve full simulation for validation, which we leave to future work.

5 Case Study

In this section, we examine the design and agent decisions that most clearly reveal the role of high-level abstractions in agentic chip design. We first show how Agent-based HLS Design can scale designs efficiently, illustrating how abstractions expose useful design knowledge. We then show that compilers translating these abstractions can still leave inefficiencies that agents can identify and improve. Finally, we examine how Post-HLS RTL Refinement can improve timing and area beneath the HLS-generated design.

5.1 High Level Abstractions Distill Design Knowledge for Agent

Figure 4 shows ROB1, greedy non-maximum suppression. Object detectors often produce multiple high-scoring, overlapping candidate boxes for the same object. Non-maximum suppression (NMS) reduces these duplicates by repeatedly selecting the highest-scoring remaining box and suppressing candidates whose intersection-over-union (IoU) with it exceeds a threshold [3]. ROB1 applies greedy NMS to up to 8192 candidate boxes, keeps at most 512, and uses a 50% IoU threshold. For each retained box, the design performs an argmax pass to select the next box and a suppression pass to remove overlapping candidates. This requires roughly $2KN$ box visits for K retained boxes among N candidates, making the number of boxes processed per cycle the main latency bottleneck.

With HLS, the agent asked for that degree of parallelism in one pragma each, an `ARRAY_PARTITION` into 256 banks and a `PIPELINE`

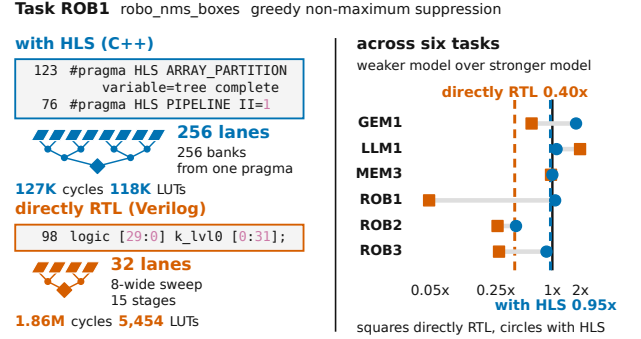


Figure 4: Abstractions distill design knowledge for agents. On the left, the same agent implements ROB1 through HLS and direct RTL Design. The HLS design uses two pragmas to produce 256-way parallelism, while the direct RTL design achieves only 32/8-way parallelism despite requiring more Verilog than HLS C++. On the right, across six tasks, the weaker model nearly matches the stronger model when both operate through HLS.

at one iteration per cycle, and Vitis HLS supplied what the request implies: a 256-port read, a log-depth comparison tree and the pipeline registers between its levels, in 43914 lines of Verilog. The same agent writing Verilog directly built a 32-lane argmax and an 8-lane suppression sweep in 378 lines and 15 pipeline stages, reading one 512-bit word per cycle from the memory port. Its design runs 1.86M cycles against 127K, with 5454 LUTs against HLS-generated RTL’s 118K. The right panel of Figure 4 shows the same effect across models: on the six tasks where all 4 pairs of (GPT/Gemini)x(HLS/RTL) succeed, the weaker model reaches 0.40x the speedup of the stronger with direct RTL and 0.95x with

HLS. This demonstrate that HLS simplify the effort of design optimization.

5.2 Agents Recover Inefficiencies Left by Compiler Heuristics

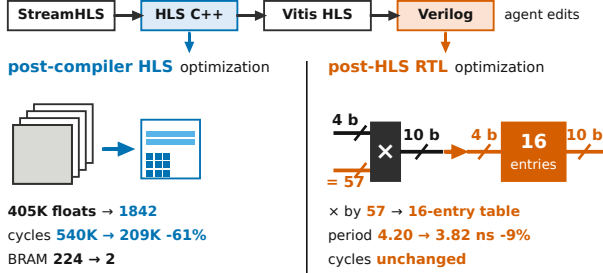


Figure 5: Agents recover inefficiencies left by compiler heuristics. The StreamHLS design passes through Post-Compiler HLS Refinement and Post-HLS RTL Refinement. At HLS level, Four whole tensors are rewritten to two rows and a window, reducing cycle count by 61%. At RTL level, a multiplier with a pinned operand becomes a lookup table, reducing the clock period by 9% at unchanged cycle count.

Figure 5 follows the SHL2 design through Post-Compiler HLS Refinement (iv) and Post-HLS RTL Refinement (iii). StreamHLS generates four local arrays totaling 405K floating-point values and adds an extra stage to fill a padded copy before the convolution reads it. This illustrates a common limitation of source-to-source compilers: even when the abstraction points optimization in the right direction, code generation still relies on fixed heuristics that may not generalize across designs.

This is where agents can complement compiler automation. With Post-Compiler HLS Refinement, the agent kept two rows of line buffer and a 3×3 window, 1842 floating points in all, and deleted the padding stage. The cycle count fell from 539726 to 209297 and the block RAM from 224 to 2.

The same design also benefits from Post-HLS RTL Refinement. The agent identifies two multiplier modules whose second operands are already constant in the generated RTL and replaces them with lookup tables. The clock period drops from 4.20 to 3.82 ns with no change in cycle count.

This case illustrates an ideal division of optimization, where compilers and higher-level abstractions make the “giant steps” and agents recover case-specific “baby steps” with surgical precision.

5.3 Agents optimize timing and area beneath the HLS schedule

Figure 6 traces two designs through Post-HLS RTL Refinement (iii). At the HLS level, both computations are expressed compactly, but the generated RTL exposes lower-level implementation choices that the agent can further improve.

On MEM3, the HLS design computes $\max(q \cdot \min, q \cdot \max)$ for each of 32 vector elements. Vitis HLS implements this with two multipliers per element and selects the larger product, using 64 DSPs in total. Gemini observes that the sign of q alone determines

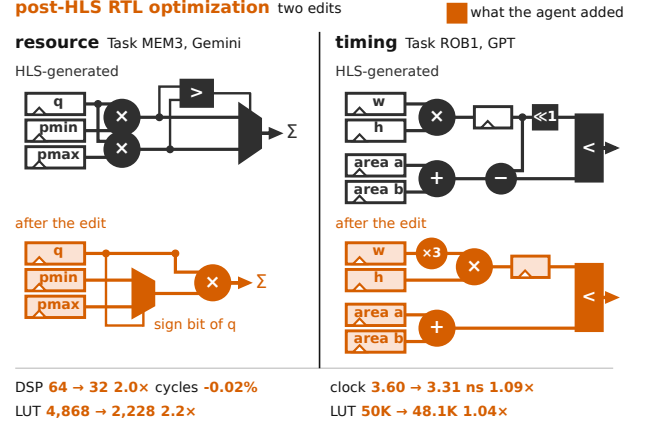


Figure 6: Agents optimize timing and area beneath the HLS schedule: on MEM3, moving the min/max selection before multiplication cuts DSP usage and cuts LUTs by 2.2x without changing the cycle count. On ROB1, rewriting the comparison removes a subtractor from the critical path and reduces the clock period from 3.60 to 3.31 ns.

whether $\min$ or $\max$ should be used, and moves this selection before the multiplication. This reduces the design to one multiplier per element, cutting DSP usage from 64 to 32 and LUT usage from 4,868 to 2,228, while keeping the cycle count nearly unchanged. The clock period also improves from 3.14 to 2.83 ns.

On ROB1, the generated RTL implements the 50% intersection-over-union comparison as $\text{union} < 2 \cdot \text{intersection}$, where $\text{union} = a + b - \text{intersection}$. This places a multiplier, a 33-bit subtractor, and a comparator on the critical path. GPT instead rewrites the equivalent condition as $a + b < 3 \cdot \text{intersection}$, removing the subtractor from the path. The clock period improves from 3.60 to 3.31 ns, meeting the 3.33 ns target, with the same cycle count and 4% fewer LUTs.

These examples show another useful division of optimization. HLS captures the high-level computation and generates a complete implementation, while the agent can inspect the resulting RTL and recover case-specific opportunities in resource mapping, arithmetic structure, and timing that only become explicit after RTL generation.

5.4 Recommended Flow

Our results suggest combining Agent-based HLS Design with Post-HLS RTL Refinement as a promising flow for agentic chip design, which means that starting with a design specification in natural language or some high-level executable language, an agent generates optimized HLS C/C++ code, and the further inspects the generated RTL code by an HLS tool for further refinement at the RTL level. We name it the AHRR Flow. Figure 7 illustrates the AHRR flow.

The key strength of AHRR is its combination of *architectural leverage* from HLS and *cycle-accurate control* at RTL. At the HLS level, the agent can efficiently explore broad architectural transformations through compact C/C++ changes and compiler directives. After RTL generation, the same agent can inspect the generated implementation and further refine timing, storage, and datapath decisions that only become explicit at the lower level. These two stages are complementary: Agent-based HLS Design already achieves a

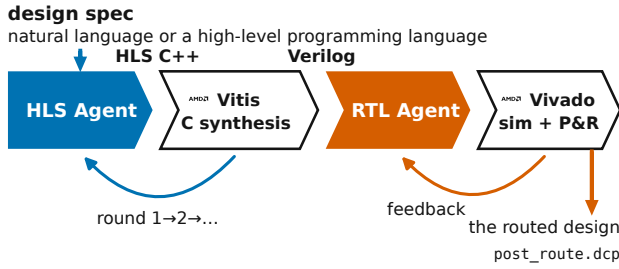


Figure 7: The proposed AHRR workflow.

2.31× geometric-mean speedup over Direct RTL Design, while subsequent RTL refinement increases the speedup to 2.62×. This ability to operate effectively on both sides of the HLS compiler boundary makes AHRR particularly promising for agentic chip design.

6 Conclusion

In this study, through extensive evaluation using the latest commercial AI agents in conjunction with state-of-the-art HLS tools, we conclude that the AHRR flow proposed in the preceding section is particularly promising for high-quality RTL design generation. The key strength of this approach is its combination of **machine intelligence**, embodied in frontier AI models, and **human intelligence**, embodied in HLS and domain-specific compilers. HLS provides a correct-by-construction mechanism for bridging the gap between behavioral specifications expressed in C/C++ or other high-level languages and the cycle-accurate implementations required for RTL designs. Meanwhile, today’s coding agents have demonstrated remarkable capabilities in high-level programming, leveraging the vast amount of software code represented in their training data, as well as an impressive ability to understand and refine RTL generated by HLS compilers. The combination of these complementary capabilities makes the AHRR flow particularly powerful in the agentic era.

It is worth noting that, despite the significant productivity gains offered by HLS, HLS-based design methodologies have historically seen limited adoption, due in part to two barriers: (i) the inertia of existing RTL-based design practices, which makes designers reluctant to adopt a new methodology, and (ii) the difficulty for human designers to read and understand HLS-generated RTL when cycle-accurate modifications are required. AI agents can overcome both barriers. Unlike human designers, agents can readily explore and adopt HLS-based methodologies when they produce better results, without being constrained by the inertia of established design practices. Moreover, agents can effectively understand and further refine the RTL generated by HLS tools when low-level control is necessary. We therefore expect agents to substantially accelerate the adoption of HLS and other high-level design methodologies. More broadly, our results suggest a promising paradigm for chip design: **agents can design better chips by leveraging tools that operate at higher levels of abstraction**. A similar view was also expressed in a recent position paper [33].

As we completed this study, it is encouraging to see that AI agent + HLS design is being adopted by some major industry players, such as the Jalapeño chip designed by OpenAI and announced in August

2026 [16]. No details on the design flow has been released yet, but it is an exciting development.

Acknowledgments

This work was partially supported by NSF grants DGE-2034835, SRC JUMP 2.0 PRISM Center, Google, Jane Street, and the CDSC industrial partners (<https://cdsc.ucla.edu/partners/>). The authors thank AMD/Xilinx for HACC equipment donation and HPCfund cluster access. J. Cong has a financial interest in AMD and Google.

References

- [1] Yunsheng Bai, Ghaith Bany Hamad, Chia-Tung Ho, Syed Suhaib, and Haoxing Ren. 2025. FVDebug: An LLM-Driven Debugging Assistant for Automated Root Cause Analysis of Formal Verification Failures. *arXiv preprint arXiv:2510.15906* (2025).
- [2] Suhail Basalama and Jason Cong. 2025. Stream-HLS: Towards Automatic Dataflow Acceleration. *arXiv e-prints* (2025), arXiv–2501.
- [3] Navaneeth Bodla, Bharat Singh, Rama Chellappa, and Larry S Davis. 2017. Soft-NMS—Improving object detection with one line of code. In *2017 IEEE international conference on computer vision (ICCV)*. IEEE, 5562–5570.
- [4] Hongzheng Chen, Niansong Zhang, Shaojie Xiang, Zhichen Zeng, Mengjia Dai, and Zhiru Zhang. 2024. Allo: A programming model for composable accelerator design. *Proceedings of the ACM on Programming Languages* 8, PLDI (2024), 593–620.
- [5] Yuze Chi, Jason Cong, Peng Wei, and Peipei Zhou. 2018. SODA: Stencil with optimized dataflow architecture. In *Proceedings of the International Conference on Computer-Aided Design*. 1–8.
- [6] Luca Collini, Siddharth Garg, and Ramesh Karri. 2024. C2HLS: Can LLMs Bridge the Software-to-Hardware Design Gap? *arXiv preprint arXiv:2406.09233* (2024).
- [7] Jason Cong, Jason Lau, Gai Liu, Stephen Neuendorffer, Peichen Pan, Kees Vissers, and Zhiru Zhang. 2022. FPGA HLS today: successes, challenges, and opportunities. *ACM Transactions on Reconfigurable Technology and Systems (TRETS)* 15, 4 (2022), 1–42.
- [8] Jason Cong, Bin Liu, Stephen Neuendorffer, Juanjo Noguera, Kees Vissers, and Zhiru Zhang. 2011. High-level synthesis for FPGAs: From prototyping to deployment. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 30, 4 (2011), 473–491.
- [9] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, et al. 2024. The Llama 3 Herd of Models. arXiv:2407.21783 [cs.AI] <https://arxiv.org/abs/2407.21783>
- [10] Chia-Tung Ho, Haoxing Ren, and Bruce Khailany. 2024. Verilogcoder: Autonomous verilog coding agents with graph-based planning and abstract syntax tree (AST)-based waveform tracing tool. *arXiv preprint arXiv:2408.08927* (2024).
- [11] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. arXiv:2309.06180 [cs.LG] <https://arxiv.org/abs/2309.06180>
- [12] Alex H. Lang, Sourabh Vora, Holger Caesar, Lubing Zhou, Jiong Yang, and Oscar Beijbom. 2019. PointPillars: Fast Encoders for Object Detection from Point Clouds. arXiv:1812.05784 [cs.LG] <https://arxiv.org/abs/1812.05784>
- [13] Mingjie Liu, Nathaniel Pinckney, Bruce Khailany, and Haoxing Ren. 2023. VerilogEval: Evaluating Large Language Models for Verilog Code Generation. arXiv:2309.07544 [cs.LG] <https://arxiv.org/abs/2309.07544>
- [14] Mario Zechner and contributors. 2026. pi: AI agent toolkit: unified LLM API, agent loop, TUI, coding agent CLI. <https://github.com/badlogic/pi-mono>. Accessed 2026-09-03. Page title as fetched: GitHub - earendil-works/pi: AI agent toolkit: unified LLM API, agent loop, TUI, coding agent CLI - GitHub.
- [15] Alexander Novikov, Ngan Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco JR Ruiz, Abbas Mehrabian, et al. 2025. Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131* (2025).
- [16] OpenAI. 2026. OpenAI Jalapeño. <https://openai.com/index/jalapeno-first-results/>.
- [17] Nathaniel Pinckney, Chenhui Deng, Chia-Tung Ho, Yun-Da Tsai, Mingjie Liu, Wenfei Zhou, Bruce Khailany, and Haoxing Ren. 2025. Comprehensive Verilog design problems: A next-generation benchmark dataset for evaluating large language models and agents on rtl design and verification. *arXiv preprint arXiv:2506.14074* (2025).
- [18] Stéphane Pouget, Michael Lo, Louis-Noël Pouchet, and Jason Cong. 2025. Holistic Optimization Framework for FPGA Accelerators. *ACM Transactions on Design Automation of Electronic Systems* 31, 1 (2025), 1–37.
- [19] Neha Prakriya, Zijian Ding, Yizhou Sun, and Jason Cong. 2025. LIFT: LLM-based pragma insertion for HLS via GNN supervised fine-tuning. *arXiv preprint arXiv:2504.21187* (2025).

- [20] Stephen Robertson and Hugo Zaragoza. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and trends® in information retrieval* 4, 1-2 (2009), 1–174.
- [21] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2024. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems* 36 (2024).
- [22] Shailja Thakur, Baleegh Ahmad, Hammond Pearce, Benjamin Tan, Brendan Dolan-Gavitt, Ramesh Karri, and Siddharth Garg. 2024. Verigen: A large language model for verilog code generation. *ACM Transactions on Design Automation of Electronic Systems* 29, 3 (2024), 1–31.
- [23] Hanyu Wang, Xinrui Wu, Zijian Ding, Su Zheng, Chengyue Wang, Neha Prakriya, Tony Nowatzki, Yizhou Sun, and Jason Cong. 2025. LLM-DSE: Searching Accelerator Parameters with LLM Agents. *arXiv preprint arXiv:2505.12188* (2025).
- [24] Jie Wang, Licheng Guo, and Jason Cong. 2021. AutoSA: A polyhedral compiler for high-performance systolic arrays on FPGA. In *The 2021 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*. 93–104.
- [25] Chenwei Xiong, Cheng Liu, Huawei Li, and Xiaowei Li. 2024. HLSPilot: LLM-based High-Level Synthesis. *arXiv:2408.06810 [cs.AR]* <https://arxiv.org/abs/2408.06810>
- [26] Kangwei Xu, Grace Li Zhang, Xunzhao Yin, Cheng Zhuo, Ulf Schlichtmann, and Bing Li. 2025. HLSRewriter: Efficient Refactoring and Optimization of C/C++ Code with LLMs for High-Level Synthesis. *ACM Transactions on Design Automation of Electronic Systems* (2025).
- [27] Shang Yang, Junxian Guo, Haotian Tang, Qinghao Hu, Guangxuan Xiao, Jiaming Tang, Yujun Lin, Zhijian Liu, Yao Lu, and Song Han. 2025. LServe: Efficient Long-sequence LLM Serving with Unified Sparse Attention. *arXiv:2502.14866 [cs.CL]* <https://arxiv.org/abs/2502.14866>
- [28] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629* (2022).
- [29] Hanchen Ye and Deming Chen. 2025. StreamTensor: Make Tensors Stream in Dataflow Accelerators for LLMs. In *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*. 201–216.
- [30] Hanchen Ye, Cong Hao, Jianyi Cheng, Hyunmin Jeong, Jack Huang, Stephen Neuendorffer, and Deming Chen. 2022. ScaleHLS: A new scalable high-level synthesis framework on multi-level intermediate representation. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 741–755.
- [31] Hanchen Ye, Hyegang Jun, and Deming Chen. 2024. HIDA: A Hierarchical Dataflow Compiler for High-Level Synthesis. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 215–230.
- [32] Jiahao Zhang, Zifan He, Nicholas Fraser, Michaela Blott, Yizhou Sun, and Jason Cong. 2026. FlexLLM: Composable HLS Library for Flexible Hybrid LLM Accelerator Design. *arXiv preprint arXiv:2601.15710* (2026).
- [33] Niansong Zhang, Sunwoo Kim, Shreesha Srinath, and Zhiru Zhang. 2026. From Pragmas to Partners: A Symbiotic Evolution of Agentic High-Level Synthesis. *arXiv preprint arXiv:2602.01401* (2026).
- [34] Yujie Zhao, Hejia Zhang, Hanxian Huang, Zhongming Yu, and Jishen Zhao. 2025. MAGE: A multi-agent engine for automated rtl code generation. In *2025 62nd ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–7.
- [35] Yang Zou, Zijian Ding, Yizhou Sun, and Jason Cong. 2026. AgRefactor: Self-Evolving Agentic Workflow for HLS Compatibility and Performance. *arXiv preprint arXiv:2606.30949* (2026).